



Universidad Nacional Autónoma de México



Facultad de Ingeniería

Alumno:

Lara Hernandez Angel Husiel - 320060829

Negocios electronicos y Desarrollo Web.

Grupo: 03 - Semestre: 2027-1

Tarea 2:

Patrones de Diseño.

Profesor:

MTRO. GERARDO GABRIEL CARRASCO ZUÑIGA

Fecha de Entrega:

6 de Octubre del 2026



Índice

1 Configuración del Entorno en Google Colab	1
Preguntas - Factory	1
2 Patrón Strategy - Estrategias de Descuento	2
Preguntas - Strategy	3
3 Patrón Observer (Notificaciones)	4
Preguntas - Observer	5
4 Patrón Adapter	5
Preguntas - Adapter	6
5 Integración de Mini-Checkout	7
6 Pruebas y Validación del Checkout	8
Tabla Comparativa	10
7 Preguntas de Validación Final	10
Conclusión	12



1. Configuración del Entorno en Google Colab

Ejecutar el siguiente comando:

```
1 !pip -q install flask pyngrok
```

Patrón Factory (crear el procesador de Pago)

Propósito: Lo que hace este patrón creacional es dejar la creación de los objetos en una sola función, donde en este caso `crear_pago(tipo)` recibe el tipo de pago y nos regresa un procesador de Stripe o PayPal listo para cobrar. Esto para que el resto del programa solo utilice el método `cobrar()` sin tener que crear directamente cada clase en todas las partes del código.

Implementación:

```
1 # Celda 2 - Patrón Factory: Procesadores de pago
2 class PagoStripe:
3     def cobrar(self, monto):
4         print(f'[Stripe] Procesando pago de ${monto:.2f}')
5         return True
6
7 class PagoPayPal:
8     def cobrar(self, monto):
9         print(f'[PayPal] Procesando pago de ${monto:.2f}')
10        return True
11
12 def crear_pago(tipo):
13     """Factory: crea el procesador de pago según el tipo recibido."""
14     fabricas = {
15         'stripe': PagoStripe,
16         'paypal': PagoPayPal
17     }
18     if tipo not in fabricas:
19         raise ValueError(f'Tipo de pago no soportado: {tipo}')
20     return fabricas[tipo]()
21
22 # Prueba rápida
23 procesador = crear_pago('stripe')
24 procesador.cobrar(100) # Salida: [Stripe] Procesando pago de $100.00
```

Salida:

```
[Stripe] Procesando pago de $100.00
True
```

Preguntas - Factory



1. ¿Por qué es preferible usar una función `crear_pago()` en lugar de instanciar directamente `PagoStripe()` o `PagoPayPal()` en cada parte del código?

Esto para poder tener un mejor control de la creación de los objetos, además de revisar si el tipo de pago está soportado. Si instanciamos directamente, el código queda acoplado a esas clases, por lo que tendríamos que regresar a modificar varias partes cuando agregamos un nuevo método de pago. Factory nos ayuda con el principio de Abierto/Cerrado (OCP) en el código que utiliza los pagos, donde podemos agregar otro procesador sin cambiar el flujo principal. No obstante, en esta implementación si se tiene que actualizar el registro del diccionario `fabricas`.

2. ¿Qué tendría que cambiar si quisieras agregar soporte para `MercadoPago` como tercer método de pago?

Solo se tendría que crear la clase `PagoMercadoPago` con su método `cobrar(self, monto)`, de igual forma con el mismo parámetro del monto, y registrar esa clase con la clave `mercadopago` en el diccionario `fabricas` dentro de la función de creación de pago. De esta forma `crear_pago('mercadopago')` ya podría regresar ese procesador sin cambiar la forma en que el checkout llama a `cobrar()`.

3. ¿Qué excepción lanza la función si se pasa un tipo de pago desconocido? ¿Cómo podrías manejar ese error en el flujo principal?

Se lanza la excepción `ValueError`, esto por que el tipo de pago no se encuentra registrado en el diccionario `fabricas`. No es solo un mensaje de `print`, ya que si no se maneja la excepción, se detiene la ejecución de esa llamada. Para manejarla se puede poner la llamada a `crear_pago()` o a `flujo_checkout()` dentro de un bloque `try/except ValueError`, donde se muestre el mensaje del error y se le pida al usuario elegir un método válido, sin continuar con el cobro de ese intento.

2. Patrón Strategy - Estrategias de Descuento

Propósito: Lo que hace este patrón de comportamiento es dejar cada descuento en su propia clase, donde el carrito solo llama al método `aplicar()` de la estrategia que tenga en ese momento. Esto para poder cambiar la forma de calcular el descuento sin modificar la clase `Carrito`.

Implementación:

```
1 # Celda 3 - Patrón Strategy: Descuentos
2
3 class DescuentoVIP:
4     def aplicar(self, monto):
5         descuento = monto * 0.15
6         print(f'[VIP] Descuento del 15%: -${descuento:.2f}')
7         return monto - descuento
8
9 class DescuentoBlackFriday:
10     def aplicar(self, monto):
11         descuento = monto * 0.30
12         print(f'[Black Friday] Descuento del 30%: -${descuento:.2f}')
13         return monto - descuento
14
```



```
15 class SinDescuento:
16     def aplicar(self, monto):
17         return monto # Sin cambios al precio
18
19 class Carrito:
20     def __init__(self, estrategia_descuento=None):
21         self.estrategia = estrategia_descuento or SinDescuento()
22
23     def calcular_total(self, monto_base):
24         return self.estrategia.aplicar(monto_base)
25
26 # Pruebas de estrategias
27 carrito_vip = Carrito(DescuentoVIP())
28 print('Total VIP:', carrito_vip.calcular_total(200)) # 170.0
29
30 carrito_bf = Carrito(DescuentoBlackFriday())
31 print('Total Black Friday:', carrito_bf.calcular_total(200)) # 140.0
```

Salida:

```
[VIP] Descuento del 15%: -$30.00
Total VIP: 170.0
[Black Friday] Descuento del 30%: -$60.00
Total Black Friday: 140.0
```

Preguntas - Strategy

4. ¿Cómo cambiarías la estrategia de descuento en tiempo de ejecución, después de haber creado el Carrito?

Pues una vez creado el carrito, solo se debe de reasignar su atributo `estrategia` con una nueva instancia, donde en este caso seria `carrito_bf.estrategia = DescuentoVIP()`. Esto reemplaza el descuento de Black Friday por el VIP, por lo que no se suman los dos descuentos. Al volver a llamar a `calcular_total(200)`, el carrito utiliza la estrategia actual y nos regresa 170. No se vuelve a ejecutar el constructor, ya que lo que cambia es el atributo del objeto que ya tenemos creado.

```
1 carrito_bf.estrategia = DescuentoVIP()
2 print('Total despues de cambiar a VIP:', carrito_bf.calcular_total(200))
```

Salida:

```
[VIP] Descuento del 15%: -$30.00
Total despues de cambiar a VIP: 170.0
```

5. ¿Qué ventaja tiene encapsular cada descuento en su propia clase en lugar de usar un bloque if/elif en el método `calcular_total()`?

Lo que hacemos al encapsular los descuentos es que cada clase tenga una sola responsabilidad, por lo que podemos probar y cambiar cada regla por separado. Además de esto, se evita que el método `calcular_total()` crezca con un bloque de if/elif cada vez que agregamos una promoción. Esto nos ayuda con Responsabilidad Única y Abierto/Cerrado, ya que podemos añadir otro descuento sin modificar la clase `Carrito`.



6. Crea una nueva estrategia DescuentoEstudiante que aplique un 10 % de descuento e intégrala al código existente.

En este caso creamos la clase `DescuentoEstudiante` con el mismo metodo `aplicar(self, monto)`, donde se calcula el 10 % del monto y se regresa el precio despues de quitar ese descuento. De igual forma la pasamos al `Carrito`, por lo que con un producto de 200 nos queda un total de 180. Ademas de esto, se registra con la clave `estudiante` en el flujo del checkout para poder utilizarla junto con los pagos, el envio y las notificaciones.

```
1 # Pregunta 6 - Nueva estrategia de descuento del 10%
2 class DescuentoEstudiante:
3     def aplicar(self, monto):
4         descuento = monto * 0.10
5         print(f'[Estudiante] Descuento del 10%: -${descuento:.2f}')
6         return monto - descuento
7
8 # Se utiliza el mismo Carrito, sin cambiar su implementacion.
9 carrito_estudiante = Carrito(DescuentoEstudiante())
10 total_estudiante = carrito_estudiante.calcular_total(200)
11 assert total_estudiante == 180.0
12 print('Total estudiante:', total_estudiante)
```

Salida:

```
[Estudiante] Descuento del 10%: -$20.00
Total estudiante: 180.0
```

3. Patron Observer (Notificaciones)

Propósito: Lo que hace este patron es permitir que un objeto avise a varios objetos cuando ocurre un evento, donde en este caso tenemos al `Notificador`, que se encarga de avisar a los observadores que se encuentran suscritos.

```
1 # Celda 4 - Patrón Observer: Notificaciones
2
3 class Notificador:
4     """Sujeto observable que mantiene la lista de suscriptores."""
5     def __init__(self):
6         self._observadores = []
7
8     def suscribir(self, observador):
9         self._observadores.append(observador)
10
11     def desuscribir(self, observador):
12         self._observadores.remove(observador)
13
14     def notificar(self, evento):
15         for obs in self._observadores:
16             obs.actualizar(evento)
17
18 class NotificadorCorreo:
19     """Observador: envía notificación por correo electrónico."""
```



```
20     def actualizar(self, evento):
21         print(f'[Email] Notificación enviada al cliente: {evento}')
22
23     class GeneradorFactura:
24         """Observador: genera la factura electrónica."""
25         def actualizar(self, evento):
26             print(f'[Factura] Generando factura para: {evento}')
27
28     # Prueba
29     notificador = Notificador()
30     notificador.suscribir(NotificadorCorreo())
31     notificador.suscribir(GeneradorFactura())
32     notificador.notificar('Pago de $170.00 con Stripe')
```

Salida:

```
[Email] Notificación enviada al cliente: Pago de $170.00 con Stripe
[Factura] Generando factura para: Pago de $170.00 con Stripe
```

Preguntas - Observer

7. ¿Qué ocurre si llamas a `notificador.notificar()` sin haber suscrito ningún observador?

Pues en este caso no ocurre ningún error, esto por que la lista de observadores se encuentra vacía, por lo que el ciclo `for` no tiene ningún objeto que recorrer. Lo que hace es que el programa siga normalmente, pero no se manda ninguna notificación, ya que no hay nadie suscrito para recibirla.

8. ¿Cómo implementarías un observador `SMSNotificador` que imprima `[SMS] Mensaje enviado: {evento}`?

Solo se tendría que crear la clase `SMSNotificador`, de igual forma como se hizo con el correo y la factura, donde esta clase debe de tener el método `actualizar(self, evento)` y dentro poner `print(f'[SMS] Mensaje enviado: {evento}')`. Además de esto, para que reciba los avisos se tendría que registrar con `notificador.suscribir(SMSNotificador())`, lo que hace que se agregue a la lista sin modificar la clase `Notificador`.

9. ¿Por qué se dice que el Observer desacopla al emisor de los receptores del evento?

Esto por que el `Notificador` solo se encarga de recorrer la lista de los observadores y llamar al método `actualizar(evento)` de cada uno, donde no necesita saber como se manda el correo o como se genera la factura. Cada clase se encarga de su propia parte, por lo que si en un futuro queremos agregar las notificaciones por SMS, solo agregamos ese observador y lo suscribimos, sin cambiar la lógica del emisor.

4. Patrón Adapter

Propósito: Lo que hace este patrón es adaptar la forma en que se utiliza una clase para que sea compatible con lo que necesita nuestro sistema.

```
1 # Celda 5 - Patrón Adapter: Cotización de envío
```



```
2
3 class APIEnvioFalsa:
4     """API externa con interfaz incompatible (en inglés)."""
5     def get_shipping_quote(self, destination, weight_kg):
6         return {
7             'cost_usd': weight_kg * 5.0,
8             'days': 3
9         }
10
11 class AdapterEnvio:
12     """Adapter: adapta APIEnvioFalsa a la interfaz esperada por el checkout."""
13     def __init__(self, api_externa):
14         self._api = api_externa
15
16     def cotizar_envio(self, destino, peso_kg):
17         resultado = self._api.get_shipping_quote(destino, peso_kg)
18         costo = resultado['cost_usd']
19         dias = resultado['days']
20         print(f'[Envío] Destino: {destino} | Costo: ${costo:.2f} | Días: {dias}')
21         return costo
22
23 # Prueba
24 api_externa = APIEnvioFalsa()
25 adapter = AdapterEnvio(api_externa)
26 costo_envio = adapter.cotizar_envio('Ciudad de México', 2.0)
27 # Salida: [Envío] Destino: Ciudad de México | Costo: $10.00 | Días: 3
```

Salida:

```
[Envío] Destino: Ciudad de México | Costo: $10.00 | Días: 3
```

Preguntas - Adapter

10. ¿Por qué no es recomendable llamar directamente a `get_shipping_quote()` desde el flujo principal del checkout?

Esto por que el checkout quedaria dependiendo directamente de la API externa, donde si en un futuro cambia el nombre del metodo o los parametros que recibe, tendríamos que modificar el flujo principal. Lo que hacemos al utilizar el Adapter es dejar esos cambios en una sola clase, por lo que se nos hace mas facil cambiar el proveedor de envio sin modificar toda la parte de la compra.

11. ¿Qué cambiarías en `AdapterEnvio` si la `APIEnvioFalsa` comenzara a devolver el costo en pesos mexicanos (MXN) en lugar de dólares?

Solo se tendria que modificar el metodo `cotizar_envio()` dentro de `AdapterEnvio`, donde primero se toma el costo en pesos de la respuesta de la API y se convierte a dolares. De esta forma el checkout sigue recibiendo el costo en la moneda que espera.



12. ¿Cuál es la diferencia principal entre el Adapter y el Decorator como patrones estructurales?

En el caso de Adapter se usa mas que nada para hacer compatibles dos interfaces que son distintas, como fue el caso de la API de envio con nuestro checkout. En el caso del Decorator, lo que hace es agregar funciones o comportamientos a un objeto conservando la misma interfaz, por lo que uno nos ayuda a adaptar como se usa una clase y el otro a agregarle mas funcionalidades.

5. Integración de Mini-Checkout

Ahora bien, una vez que tenemos los cuatro patrones por separado, lo que hacemos es juntarlos en la funcion `flujo_checkout()`. Primero se utiliza Factory para crear el metodo de pago, despues Strategy para aplicar el descuento al monto del producto y de igual forma Adapter para obtener el costo del envio, donde este costo se suma al total que ya tenia el descuento.

Por ultimo se realiza el cobro y, si este fue exitoso, se utiliza Observer para avisar al correo y al generador de la factura.

```
1  # Celda 6 - Flujo completo del mini-checkout
2
3  def flujo_checkout(tipo_pago, monto_producto,
4                    tipo_descuento='ninguno',
5                    destino='Ciudad de México',
6                    peso_kg=1.0):
7      print('\n' + '='*50)
8      print('          INICIANDO CHECKOUT')
9      print('='*50)
10
11     # 1. Factory: crear procesador de pago
12     procesador = crear_pago(tipo_pago)
13
14     # 2. Strategy: calcular total con descuento
15     estrategias = {
16         'vip':          DescuentoVIP(),
17         'black_friday': DescuentoBlackFriday(),
18         'estudiante':   DescuentoEstudiante(),
19         'ninguno':      SinDescuento()
20     }
21     carrito = Carrito(estrategias.get(tipo_descuento, SinDescuento()))
22     total   = carrito.calcular_total(monto_producto)
23
24     # 3. Adapter: cotizar envío
25     api_envio = APIEnvioFalsa()
26     adapter   = AdapterEnvio(api_envio)
27     costo_envio = adapter.cotizar_envio(destino, peso_kg)
28     total_final = total + costo_envio
29     print(f'[Total con envío] ${total_final:.2f}')
30
31     # 4. Observer: notificar al pagar
32     notificador = Notificador()
33     notificador.suscribir(NotificadorCorreo())
34     notificador.suscribir(GeneradorFactura())
```



```
35 exito = procesador.cobrar(total_final)
36 if exito:
37     notificador.notificar(
38         f'Pago de ${total_final:.2f} con {tipo_pago.capitalize()}'
39     )
40
41     print('='*50)
42     return exito
43
44 # Prueba de ejecución integrada
45 flujo_checkout('stripe', 200.0, 'vip', 'Ciudad de México', 2.0)
```

Salida:

```
=====
                INICIANDO CHECKOUT
=====
[VIP] Descuento del 15%: -$30.00
[Envío] Destino: Ciudad de México | Costo: $10.00 | Días: 3
[Total con envío] $180.00
[Stripe] Procesando pago de $180.00
[Email] Notificación enviada al cliente: Pago de $180.00 con Stripe
[Factura] Generando factura para: Pago de $180.00 con Stripe
=====
True
```

6. Pruebas y Validación del Checkout

```
1 # Celda 7 - Pruebas y validación
2
3 # -- Prueba 1: Assertion básico del enunciado -----
4 assert crear_pago('stripe').cobrar(100)
5 print('✓ Checkout OK')
6
7 # -- Prueba 2: Flujo Stripe + descuento VIP -----
8 resultado1 = flujo_checkout(
9     tipo_pago      = 'stripe',
10    monto_producto = 200,
11    tipo_descuento  = 'vip',
12    destino         = 'Monterrey',
13    peso_kg         = 1.5
14 )
15 assert resultado1, 'Error: el flujo Stripe+VIP falló'
16 print('✓ Prueba Stripe + VIP aprobada')
17
18 # -- Prueba 3: Flujo PayPal + Black Friday -----
19 resultado2 = flujo_checkout(
20     tipo_pago      = 'paypal',
21    monto_producto = 500,
22    tipo_descuento  = 'black_friday',
23    destino         = 'Guadalajara',
24    peso_kg         = 3.0
25 )
```



```
26 assert resultado2, 'Error: el flujo PayPal+BlackFriday falló'
27 print('✓ Prueba PayPal + Black Friday aprobada')
28
29 # -- Prueba 4: Tipo de pago inválido (debe lanzar ValueError) -----
30 try:
31     crear_pago('bitcoin')
32     print('× Error: debió lanzar ValueError')
33 except ValueError as e:
34     print(f'✓ Error manejado correctamente: {e}')
35
36 print('\n✓ Todas las pruebas pasaron correctamente')
```

Salida:

```
[Stripe] Procesando pago de $100.00
✓ Checkout OK

=====
                INICIANDO CHECKOUT
=====
[VIP] Descuento del 15%: -$30.00
[Envío] Destino: Monterrey | Costo: $7.50 | Días: 3
[Total con envío] $177.50
[Stripe] Procesando pago de $177.50
[Email] Notificación enviada al cliente: Pago de $177.50 con Stripe
[Factura] Generando factura para: Pago de $177.50 con Stripe
=====
✓ Prueba Stripe + VIP aprobada

=====
                INICIANDO CHECKOUT
=====
[Black Friday] Descuento del 30%: -$150.00
[Envío] Destino: Guadalajara | Costo: $15.00 | Días: 3
[Total con envío] $365.00
[PayPal] Procesando pago de $365.00
[Email] Notificación enviada al cliente: Pago de $365.00 con Paypal
[Factura] Generando factura para: Pago de $365.00 con Paypal
=====
✓ Prueba PayPal + Black Friday aprobada
✓ Error manejado correctamente: Tipo de pago no soportado: bitcoin

✓ Todas las pruebas pasaron correctamente
```



Tabla Comparativa

Patrón	Categoría	Problema que resuelve	Clase / Función clave	Principio SOLID principal
Factory	Creacional	Evita crear directamente cada procesador en todas las partes del código, donde una sola función se encarga de elegirlo.	<code>crear_pago(tipo)</code>	OCP (Abierto/Cerrado): permite agregar métodos de pago sin cambiar el flujo principal.
Strategy	Comportamiento	Permite cambiar el descuento sin meter todas las reglas en un bloque de <code>if/elif</code> dentro del carrito.	<code>Carrito</code> , <code>DescuentoVIP</code> , <code>DescuentoBlackFriday</code> , <code>DescuentoEstudiante</code> y <code>SinDescuento</code>	OCP (Abierto/Cerrado): se pueden agregar descuentos sin modificar la clase <code>Carrito</code> .
Observer	Comportamiento	Permite avisar a varios objetos sobre el pago, donde cada uno se encarga de lo que hace con ese aviso.	<code>Notificador</code> , <code>NotificadorCorreo</code> y <code>GeneradorFactura</code>	OCP (Abierto/Cerrado): se pueden agregar observadores sin cambiar la clase <code>Notificador</code> .
Adapter	Estructural	Hace compatible la API de envío con el método que utiliza el checkout, esto para dejar los cambios del proveedor en una sola clase.	<code>AdapterEnvio</code> y <code>cotizar_envio()</code>	SRP (Responsabilidad Única): la adaptación de la API queda en <code>AdapterEnvio</code> , separada del flujo de compra.

7. Preguntas de Validación Final

1. ¿Cuál de los cuatro patrones implementados consideras el más difícil de comprender y por qué?

Pues de estos cuatro considero que el Observer es el más difícil de comprender, esto por que hay que seguir como se relaciona el objeto que manda el aviso con todos los que lo reciben. Donde en este caso el `Notificador` no hace directamente lo del correo o la factura, solo llama al método `actualizar()` de los objetos suscritos. Además de esto, hay que tener claro que suscribir un objeto solo lo agrega a la lista, y que el aviso se manda hasta que llamamos a `notificar()`, lo que hace que sea un poco menos directo de seguir que crear el pago o aplicar un descuento.

2. ¿En qué otro escenario de negocios electrónicos podrías aplicar el patrón Observer? Describe el caso brevemente.

Se podría utilizar en el inventario de una tienda, donde si un producto se agota, el inventario manda el aviso a los objetos que están suscritos. En este caso uno se encargaría de quitarlo de los productos disponibles y otro de avisar al área de compras para pedir más al proveedor, de igual forma se podría registrar ese evento para las estadísticas. Esto para que el inventario solo se encargue de mandar el aviso y no tenga que hacer toda la lógica de cada área.



3. ¿Qué desventaja tendría usar el patrón Factory si el número de métodos de pago crece a 20 opciones distintas? ¿Cómo lo mejorarías?

La desventaja sería que el diccionario `fabricas` crecería y tendríamos que mantener el registro de todas esas clases dentro de la misma función, por lo que cada nuevo método de pago nos haría regresar a modificar esa parte. No obstante, tener 20 opciones no significa que el patrón deje de servir, mas que nada se vuelve mas trabajo mantenerlo. Para mejorarlo sacaría el diccionario a un registro separado y agregaría una función para registrar los procesadores, donde `crear_pago()` solo revise el tipo y cree el objeto que corresponde.

4. ¿Qué ocurre con el principio de Responsabilidad Única si agregas la lógica de descuento directamente dentro de `crear_pago()`?

Se estaría rompiendo el principio de Responsabilidad Única, esto por que la función ya no solo se encargaría de crear el procesador de pago, sino también de calcular los descuentos. Lo que hace esto es que ahora tengamos que modificar la misma función tanto cuando cambia un método de pago como cuando cambia una promoción, por lo que lo mejor sería dejar la creación en Factory y los descuentos en sus propias clases de Strategy.

5. ¿Cómo extenderías el sistema para que el `GeneradorFactura` envíe la factura al correo del cliente automáticamente, aprovechando los patrones ya implementados?

Lo que haría sería que `GeneradorFactura`, además de recibir el aviso del pago, tenga un notificador propio para avisar cuando la factura ya esté generada. A este se le suscribiría un nuevo observador para enviar la factura por correo, donde el evento debe de incluir el correo del cliente y la referencia al archivo generado. De esta forma primero se genera la factura y después se manda el aviso para enviarla, esto por que suscribir el correo y la factura al mismo evento de pago no asegura por si solo que el correo reciba el archivo. En el código actual solo se simulan estas acciones con `print`, por lo que para un envío real también se tendría que implementar la generación del archivo y el servicio de correo.

6. ¿Qué recomendación de arquitectura darías a un equipo que va a construir un e-commerce completo basándose en lo que aprendiste en esta práctica?

Recomendaría separar las partes de pagos, descuentos, envíos y notificaciones, esto para que cada una tenga su propia responsabilidad y sea mas fácil mantener el código. De igual forma utilizaría Factory para crear los procesadores, Strategy para las promociones, Adapter para conectar los proveedores externos y Observer para los avisos de los eventos. Además de esto, dejaría los servicios como dependencias que se le pasan al flujo, por lo que si cambia el proveedor de envío o de pago, se pueda cambiar esa parte sin tener que modificar toda la compra. Por último agregaría pruebas y manejo de errores, donde se revise que un pago fallido no se tome como una compra terminada.



Conclusión

Con esta practica podemos ver como los patrones de diseño nos ayudan a organizar un proceso que tiene varias partes, donde en este caso fue una compra con su metodo de pago, su descuento, el costo del envio y las notificaciones. Lo que hacemos es que cada clase se encargue de su propia parte, por lo que se evita juntar toda la logica en una sola funcion que despues sea dificil de modificar.

Ademas de esto, con las pruebas se puede ver que podemos utilizar Stripe o PayPal y aplicar descuentos distintos conservando el mismo flujo. De igual forma, si queremos agregar otro observador o cambiar la API de envio, tenemos una parte especifica donde hacerlo, lo que hace que en un futuro sea mas facil escalar el sistema.